

WaveSim - Real-Time Procedural Ocean Simulator

Richard (Yung-Hsiang) Wang
University of Southern California
Los Angeles, CA, USA
ywang058@usc.edu

Zhuoning Wu
University of Southern California
Los Angeles, CA, USA
zhuoning@usc.edu

Wanting Jiang
University of Southern California
Los Angeles, CA, USA
wantingj@usc.edu

Abstract—We present a real-time procedural ocean simulation framework combining wave synthesis with GPU-accelerated signed distance field ray marching. Our approach generates physically plausible ocean surfaces through iterative Gerstner wave decomposition, making it suitable for interactive applications and scientific visualization.

Index Terms—ocean simulation, ray marching, GPU rendering, procedural generation

I. INTRODUCTION

This project implements a **dynamic ocean wave simulator with a fully procedural day-night sky system**. The system runs entirely in WebGL 2.0 [1], rendering in real-time directly in web browsers without requiring specialized graphics hardware or offline preprocessing. The environment features three major integrated components:

- 1) **Procedural Ocean:** Multi-octave Gerstner wave simulation with physically-based water rendering, including Fresnel reflections, subsurface scattering, and dynamic foam generation
- 2) **Dynamic Sky System:** Complete atmosphere rendering with smooth day-night transitions, procedural starfield generation, and volumetric nebula effects
- 3) **Interactive Boat Simulation:** Multiple boat models with physics-based motion responding to wave dynamics, including realistic wake patterns and surface foam

The system demonstrates the effective combination of techniques from multiple areas of computer graphics: procedural geometry generation, noise-based texture synthesis, physically-based lighting, ray marching, and particle-based effects. All components work together to create an immersive maritime environment that transitions seamlessly from day-light ocean scenes to star-filled night skies.

A. Technical Contributions

Our implementation makes several technical contributions to real-time procedural rendering:

a) *Sky and Atmosphere System:* The sky system implements a complete day-night cycle driven by circular sun motion, controlling all atmospheric transitions through a single time parameter. It features layered color gradients using overlapping smooth transitions. The procedural starfield uses grid-based distribution with hash functions for memory-free rendering with independent per-star twinkling. Volumetric nebula effects employ Fractal Brownian Motion with radial density falloff and dual-color gradients.

b) *Ocean Wave System:* The ocean component uses a multi-octave Gerstner wave simulation with ray marching for accurate water-ray intersection. It implements Fresnel effects, subsurface scattering approximation, and real-time normal computation. An interactive camera navigation system allows exploration of the environment.

c) *Boat Simulation System:* The boat subsystem supports both SDF-based procedural models and imported OBJ geometry with a unified physics-based motion system. Boats respond to waves through bobbing, multi-axis rolling, and heading alignment. Kelvin wake patterns use multi-frequency waves with angular attenuation, while dynamic foam renders at bow waves and propeller regions. The system includes Phong lighting and environmental reflections.

II. BACKGROUND

Ocean wave simulation has long been a subject of research in computer graphics, fluid dynamics, and computational physics [2]–[5]. Real-world ocean surfaces exhibit complex, multi-scale behavior resulting from wind interaction, gravitational forces, and surface tension effects. Traditional approaches to simulating ocean dynamics fall into two primary categories: physics-based simulation and procedural generation.

Physics-based approaches solve the shallow-water equations or Navier-Stokes equations to compute wave propagation from first principles. While these methods produce high-fidelity results, they incur substantial computational cost, making real-time interaction challenging. Procedural methods, conversely, approximate ocean appearance through mathematical functions without explicit fluid simulation, trading physical accuracy for computational efficiency.

A. Mathematical and Algorithmic Methods

1) *Gerstner Waves:* While sinusoidal height field representation provides visual plausibility, Gerstner (or trochoid) waves introduce realistic orbital particle motion [6]. Gerstner waves modify the vertical displacement with horizontal offsets, producing characteristic wave crests and troughs.

2) *GPU-Accelerated Ray Marching:* Ray marching is a technique for rendering volumetric and implicit surfaces by iteratively stepping along camera rays until intersection conditions are satisfied. For rendering arbitrary implicit surfaces, signed distance fields (SDFs) [7] provide a compact representation mapping 3D space to scalar distance values. GPU

implementation of ray marching in fragment shaders enables per-pixel computation, avoiding explicit mesh generation and allowing resolution-independent rendering. This approach scales gracefully with display resolution and supports dynamic viewpoint changes without geometric recomputation.

B. Similar Online Projects and Other Related Reference Resources

Shadertoy procedural Ocean samples [8]–[10], Youtube Video [11], blogs [12].

III. METHODS

A. Procedural Ocean

Procedural ocean simulation combines multiple computational techniques. The foundation of our approach rests on wave synthesis, where individual wave components are superposed according to:

$$h(\mathbf{x}, t) = \sum_{i=1}^N A_i \sin(\mathbf{k}_i \cdot \mathbf{x} + \omega_i t + \phi_i) \quad (1)$$

where A_i represents wave amplitude, $\mathbf{k}_i$ is the wave vector, ω_i is the angular frequency, and ϕ_i is the phase offset.

The integration of Gerstner waves provides a physically plausible surface displacement through orbital motion:

$$\mathbf{p}(t) = \mathbf{p}_0 + \sum_{i=1}^N A_i (\mathbf{k}_i \times \sin(\mathbf{k}_i \cdot \mathbf{p}_0 + \omega_i t)) \quad (2)$$

$\mathbf{p}_0$: This is the initial (undisplaced) position vector of the point, typically a coordinate on a flat plane. The position of the next second is calculated by adding the total displacement to this starting point.

B. Dynamic Sky System

1) *Day-Night Cycle*: The day-night cycle is driven by a circular sun trajectory

$$(x, y, z) = (R \cos \theta, R \sin \theta, z_{\text{depth}}),$$

where the angle θ increases with time. After normalization, the vertical component $h = y$ serves as the unified control parameter for all atmospheric states:

- high h : daytime lighting and clear sky,
- mid-range h : twilight with warm horizon tones,
- low h : nighttime with stars and nebula.

Smoothstep curves ensure seamless transitions among these phases, as illustrated in Fig.1.

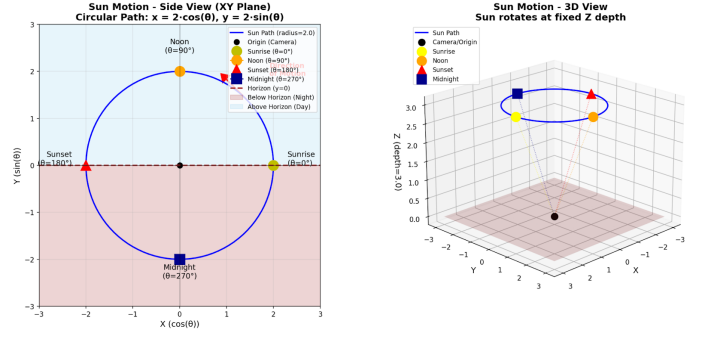


Fig. 1. Circular sun path driving sky transitions

2) *Sky Color Gradients*: Sky coloration is interpolated among three base tones representing day, twilight, and night. Gradual blending is driven by smoothstep functions, while warm twilight colors (golden, orange-red, and purple) are concentrated near the horizon through an exponential falloff:

$$w(y) = \exp(-ky^2),$$

where y is the ray direction's vertical component (ranging from 0 at the horizon to 1 at the zenith), k controls how rapidly the contribution diminishes with height, and $w(y)$ is the weight factor applied to twilight colors. This weight is multiplied with each twilight color's intensity before adding to the base sky color, causing sunset hues to appear strongest at the horizon and fade toward the zenith.

3) *Procedural Stars*: Stars are generated procedurally through a grid-based hashing workflow:

- Convert ray direction into spherical coordinates.
- Map the direction onto a discrete sky grid.
- Produce pseudo-random per-cell values to determine which cell can contain a star.
- Apply a smooth distance-based falloff from the star center, creating circular star shapes that fade from bright at the center to transparent at the edges.
- Threshold values to determine star placement.

4) *Procedural Nebula*: Nebulae are produced using multi-octave Fractal Brownian Motion (FBM) [13], [14]:

$$\text{FBM}(\mathbf{p}) = \sum_{i=0}^4 A_i \text{noise}(\mathbf{p}, F_i),$$

where $\mathbf{p} = (u, v)$ is the 2D sky coordinate, i indexes the octave number from 0 to 4, A_i is the amplitude that decreases with each octave, F_i is the frequency that increases with each octave, and $\text{noise}()$ is the value noise function. The resulting FBM values generate cloud-like structures by layering noise at multiple scales. These values are converted to density through smoothstep thresholding, spatially localized using radial falloff from a center point, and masked by night-time visibility to create the final nebula appearance with smooth opacity transitions and color gradients.

C. Interactive Boat Simulation

1) Boat Construction:

a) *Signed Distance Field*: Signed Distance Field is an implicit geometric representation where each point in space stores the signed distance to the nearest surface. Positive values indicate points outside the surface, while negative values lie inside.

b) *Smooth-min Blending*: Smooth-min function blends distance fields smoothly, avoiding the hard discontinuities produced as shown in Fig.2.



Fig. 2. Smooth-Min Example [12]

2) Shading and Reflection:

a) *Lambertian Diffuse Reflection*: Diffuse shading is modeled using the classical Lambertian formulation [15], which assumes that light scatters uniformly from a rough surface. The intensity depends solely on the cosine of the angle between the normal and the light direction.

$$D = \max(\mathbf{N} \cdot \mathbf{L}, 0) \quad (3)$$

b) *Blinn-Phong Specular Reflection*: Blinn-Phong [16] model uses the half-vector between the light direction and viewing direction to approximate the glossy specular highlights characteristic of metallic surfaces on the boat. $\mathbf{L}$ is light direction vector, $\mathbf{V}$ is View direction vector.

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|} \quad S = \max(\mathbf{N} \cdot \mathbf{H}, 0)^{\text{shininess}} \quad (4)$$

c) *Environment Mapping*: Environment mapping [17] is an image-based lighting technique that approximates specular reflection by evaluating a precomputed environment function in the direction of ideal reflection. This provides encodes the sun and sky illumination, enabling the boat's metallic surface to capture their reflected colors dynamically as the viewing direction changes, while still maintaining real-time performance without full ray tracing.

3) Motion and Orientation:

a) *Normal-Based Orientation From Surface Gradient*: Orientation derived from surface gradients relies on the fact that the normal vector of a scalar field encodes local slope. For a height field $h(x, z)$, the normal is computed from the spatial derivatives:

$$\mathbf{N} = \frac{(-\partial h / \partial x, 1, -\partial h / \partial z)}{\|(-\partial h / \partial x, 1, -\partial h / \partial z)\|} \quad (5)$$

This normal defines a local reference frame whose deviation from the global up vector provides rotational parameters (e.g., pitch and roll). The orientation may be extracted via:

$$\theta_x = \arctan\left(\frac{N_z}{N_y}\right) \quad \theta_z = \arctan\left(\frac{N_x}{N_y}\right) \quad (6)$$

which align an object smoothly with the underlying surface geometry.

b) Hierarchical Rigid-Body Transformation Matrix:

Rigid-body motion is represented using homogeneous transformation matrices. A general transform combining rotation and translation takes the form:

$$\mathbf{M} = \mathbf{T}(\mathbf{t}) \cdot \mathbf{R}(\theta_x, \theta_y, \theta_z) \cdot \mathbf{S}(\mathbf{s}) \quad (7)$$

Where $\mathbf{T}(\mathbf{t})$ is translation, $\mathbf{R}(\theta_x, \theta_y, \theta_z)$ is axis-aligned rotation matrix, $\mathbf{S}(\mathbf{s})$ is uniform or non-uniform scaling.

Matrix multiplication order determines the final orientation and is evaluated in world space. This formulation is fundamental for evaluating object poses efficiently in standard graphics pipelines.

4) Procedural Wake and Foam Modeling:

a) *Exponential & Angular Attenuation*: Attenuation functions control the spatial influence of a phenomenon. Its magnitude decays exponentially with increasing distance, and gradually diminishes as the angular deviation becomes larger:

$$A(d, \theta) = A_{\text{dist}}(d) \cdot A_{\text{angle}}(\theta) \quad (8)$$

yields a flexible shaping function for spatial patterns, allowing us to constrain the effective influence range of the wake and foam on the water surface.

b) *Procedural Noise-Based*: Based on our procedural ocean model, we incorporate Perlin noise to simulate the disturbance of the water surface caused by the boat.

D. Tools Used

We used WebGL2 for GPU-accelerated rendering and GLSL to write custom fragment and vertex shaders. JavaScript served as the high-level control layer, managing scene updates, buffer initialization, and user interaction. Finally, the simulator was deployed through GitHub Pages, enabling real-time testing and public accessibility.

E. Course Concepts Applied

- Lecture 1 Color: Gradient interpolation and perceptual blending for twilight bands.
- Lecture 2 Rays: Ray directions and spherical mapping for sky sampling.
- Lecture 3 Planes, Shading: Atmospheric scattering approximations and tone-mapping strategies. Distance-field evaluation for computing point-to-geometry relationships. The Phong illumination framework and its Blinn-Phong extension for boat.
- Lecture 5 Mesh: Standard mesh storage and attribute organization for importing boat model.
- Lecture 6 Texture Mapping: apply texture mapping to the boat model.

- Lecture 7 Procedural Generation: wave motion, hash functions, value noise, and FBM for stars and nebula.
- Lecture 8 Transform: The use of homogeneous transformation matrices for representing rigid-body motion.
- Lecture 11 Animation: time-parameterized sun motion and star twinkling cycles.

IV. EVALUATION

A. Procedural Ocean

This project generates a dynamic, multi-scale ocean surface shown in Fig.3 full rendered within the browser.

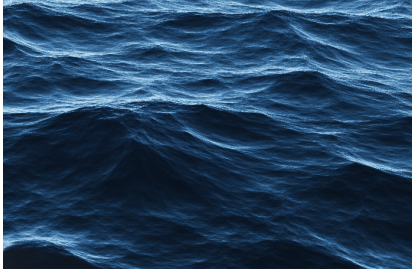


Fig. 3. Procedural Ocean Surface

Our algorithm uses the Gerstner model to achieve high-fidelity close-up wave motion. Unlike many previous approaches that separate distant and near-field effects, our method produces realistic, close-up ocean behavior within a single, efficient iterative framework.

Despite its advantages, the current solution faces several limitations inherent to procedural approaches. The simulation relies on mathematical approximations (procedural generation) rather than solving first-principles fluid dynamics, limiting its accuracy in modeling complex phenomena like wave breaking, turbulence, and realistic fluid dissipation.

B. Dynamic Sky System

1) *Day-Night Cycle*: The sun's progression through a complete day-night cycle as shown in Fig.4, demonstrates smooth continuous motion from sunrise through noon to sunset and midnight. The transition occurs over several minutes in real-time at the default rotation speed, providing natural pacing for the scene.

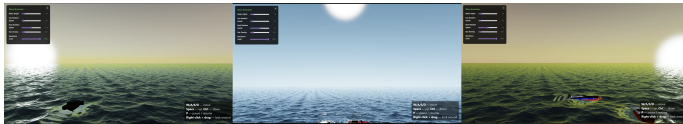


Fig. 4. Day night cycle

Our circular motion approach creates perfectly smooth transitions with no visible discontinuities, controlled by a single `SUN_ROTATION_SPEED` parameter that makes timing adjustment trivial. The sinusoidal motion is mathematically simple, producing consistent results with minimal computational

cost—only two trigonometric function calls per frame. Compared to physically-accurate astronomical simulations modeling latitude, longitude, and seasonal variations, our approach sacrifices realism for simplicity and performance, yet proves sufficient and visually convincing for artistic real-time rendering. However, the system has some limitations: the circular path lacks geographic and seasonal accuracy, with invariant cycle duration that doesn't reflect Earth's axial tilt effects. Artists have limited control over sun trajectories—custom paths like lingering sunsets require core algorithm modifications. The horizon crossing at $y = 0$ uses a hard threshold rather than atmospheric scattering, causing somewhat abrupt day-night transitions at the boundary.

2) *Sky Color Gradients*: The multi-stage color transition system creates rich atmospheric effects throughout the day-night cycle. As shown in Fig.5, the system transitions smoothly from golden sunset to pure black night. The horizon glow effect (exponential falloff) ensures twilight colors are strongest at the horizon and fade toward the zenith, matching real atmospheric scattering behavior.

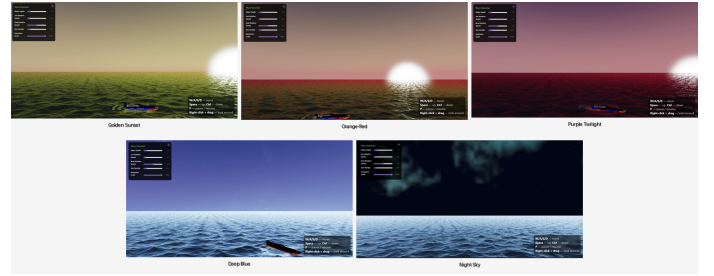


Fig. 5. Sky color cycle

The gradient system excels through overlapping color ranges that blend multiple twilight colors simultaneously (e.g., orange and purple during dusk), smoothstep interpolation for natural S-curve transitions, and exponential falloff mimicking horizon color concentration. The procedural approach uses less memory than pre-baked textures while running faster than physically-based atmospheric scattering, achieving visually similar results through simple arithmetic without texture lookups. However, limitations include simplified physics using artist-chosen colors rather than wavelength-dependent calculations, hard-coded palette requiring shader modification, lack of weather variation, flat horizon assumptions, and uniform color application across the sky hemisphere without sun-centric scattering effects.

3) *Procedural Stars*: The grid-based procedural star generation creates a realistic starfield without texture assets, demonstrating effective use of hash functions and density control.

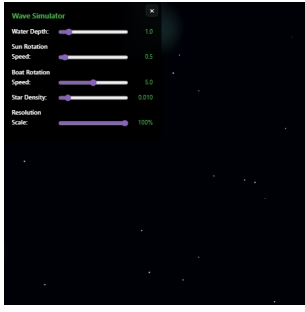


Fig. 6. Procedural stars with density = 0.01



Fig. 7. Procedural stars with density = 0.085

The procedural star system achieves zero texture memory usage, perfect resolution scalability, and temporal coherence through stable hash functions. Per-cell phase offsets create realistic asynchronous twinkling, while grid-based distribution ensures even coverage with $O(1)$ per-pixel cost versus $O(n)$ for particle systems. Runtime density control requires no recompilation. However, limitations include visible grid structure at low densities, lack of astronomical accuracy (no Milky Way clustering or constellations), uniform star sizes without magnitude variation, limited color variety, binary visibility rather than magnitude-based fading, and potential twinkling popping artifacts during state transitions.

4) *Procedural Nebula*: The FBM-based nebula system generates organic cosmic cloud structures shown in Fig.8 below.

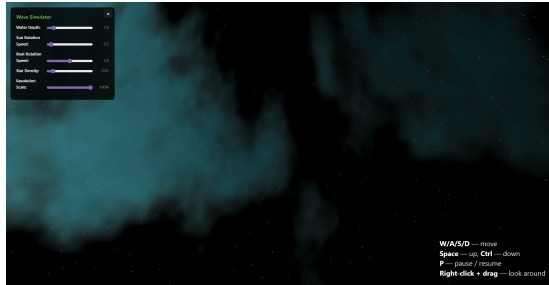


Fig. 8. Nebula rendering effect

The multi-octave FBM creates naturally irregular cloud-like structures with soft edges through smoothstep density gradients and exponential radial falloff. A second FBM layer decorrelates brightness from shape, adding realism while early exit optimization ($M \leq 0.001$) maintains performance despite 5-octave calculations. The procedural approach enables infinite variation without texture memory and renders cleanly at any resolution, outperforming texture-based and particle systems. However, limitations include static positioning without parallax, 2D screen-space lacking true depth, simplified color palette, single nebula instance, missing atmospheric extinction effects, uniform density creating less complex structures than real nebulae, and relatively high computational cost on lower-end hardware.

C. Interactive Boat Simulation



Fig. 9. SDF Boat and Model Boat

1) Boat Construction:

a) *Signed Distance Field*: The red boat in Fig.9 uses SDF. This SDF boat model is built by composing analytic distance functions of simple primitives (hull, body, nose, cockpit, windscreen) with smooth-min blending, which makes the shape easy to modify, cheap to evaluate in raymarching, and naturally provides signed distances and material IDs without needing mesh data.

b) *Real 3D Model*: The Speed Boat in Fig.9 is imported by a real 3D model. The OBJ-based boat model provides a high-fidelity mesh with detailed geometry, explicit vertex normals, and UV coordinates suitable for textured shading, enabling more realistic material appearance than the analytic SDF shape.

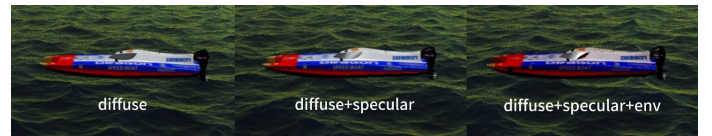


Fig. 10. Shading Phase Diagram

2) *Boat Texture and Shading*: The shading model combines diffuse lighting, Blinn-Phong specular reflection, Fresnel-based reflectance [18], and environment sampling to produce distinct material responses and view-dependent highlights. Fig.10 shows the step-by-step visual results as each lighting component is added to the boat. These components contribute to the final image by enhancing realism and allowing different boat parts to exhibit appropriate reflective or refractive behavior. The approach is highly efficient but provides only an approximate effect and lacks physical accuracy.



Fig. 11. Boat Surface Dynamics

3) *Boat Motion*: The circular path is implemented by parameterizing the boat's horizontal position with cosine and sine functions of time, producing motion along a constant-radius circle.

The boat's vertical bobbing is achieved by sampling the wave height function at the boat's current position and using that value to offset its elevation over time.

The boat's rocking motion is achieved by computing the wave-surface normal from gradient evaluation and converting its deviation from the vertical axis into pitch and roll, while the heading direction is aligned with the tangent of the circular trajectory.

After obtaining the boat's 3D position and three rotation angles, the final pose is produced through a hierarchical transformation matrix. Although damping and stiffness terms are introduced to emulate physical response, the motion remains a simplified approximation and does not reach full physical realism.

Fig.11 illustrates the resulting motion, showing the boat moving up and down along the waves while rocking naturally with the water surface.

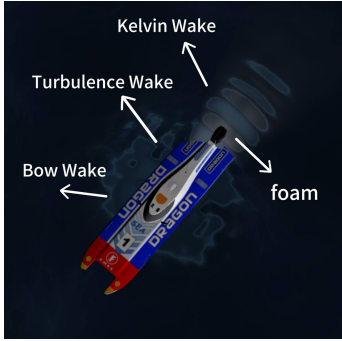


Fig. 12. Wake and Foam Diagram

4) *Wake and Foam*: The wake and foam effects generated by the boat's movement are shown in Fig.12.

a) *Wake*: The wake effect is produced by combining three components: a Kelvin wake [19] shaped through angular and distance attenuation, a bow wave [20] formed by forward-directed exponential decay, and a turbulence wake generated behind the boat. Procedural noise is applied specifically to the turbulence component to introduce stochastic variation and breakup smoothness, yielding a more natural and visually plausible turbulent trail that cannot be achieved through attenuation functions alone.

b) *Foam*: The foam effect is generated by combining directional and distance-based attenuation to shape the trailing pattern, modulating it with procedural noise to introduce turbulent variation, and amplifying its intensity using a lighting-dependent factor that increases brightness when the foam faces the sun.

c) *Speed-dependent Length*: By incorporating speed as a parameter in the attenuation functions, the system scales the spatial extent of both wake and foam, ensuring that their influence grows naturally with higher velocities. The

comparison of the wake influence range under different boat speeds is shown in Fig.13.

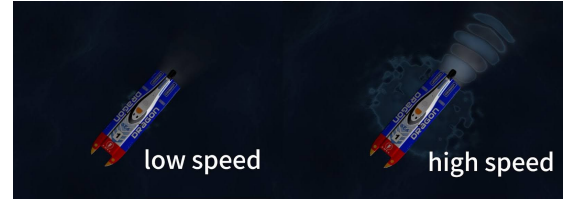


Fig. 13. Speed Comparison Diagram

V. CONCLUSION AND CONTRIBUTIONS

A. Conclusion

In this project, we developed a real-time ocean wave simulator that combines physically inspired wave motion, boat-water interaction, and a fully procedural sky system. Through building these components from scratch, we strengthened our understanding of raymarching, shading, SDF modeling, procedural noise, and time-based animation, and gained practical experience integrating multiple rendering techniques into a cohesive system.

B. Contributions

Zhuoning Wu developed the entire atmosphere and sky system, including the procedural star field, nebula generation with FBM, probability-based star placement and flickering, as well as the full day-night cycle driven by sun height and smooth color transitions. Richard Wang implemented the core ocean components, including the procedural wave simulation, the sun-color scattering effect on water, and the camera movement system used to navigate and present the scene. Wanting Jiang built the full boat subsystem for both the SDF-based and the imported 3D boat models, implementing the texture-shading pipeline, a unified motion system with circular movement, heading alignment, wave-driven bobbing and multi-axis rocking, and designing the complete wake and foam effects for both boats.

REFERENCES

- [1] Khronos WebGL Working Group. (2017) WebGL 2.0 specification. Khronos Group. [Online]. Available: <https://www.khronos.org/registry/webgl/specs/latest/2.0/>
- [2] A. Fournier and W. T. Reeves, "A simple model of ocean waves," in *Proceedings of SIGGRAPH*, 1986, pp. 75–84.
- [3] P. Janssen, *The interaction of ocean waves and wind*. Cambridge University Press, 2004.
- [4] G. J. Komen, L. Cavaleri, M. Donelan, K. Hasselmann, S. Hasselmann, P. Janssen *et al.*, *Dynamics and modelling of ocean waves*. Cambridge university press UK, 1994, vol. 532.
- [5] L. Huang, Z. Qu, X. Tan, X. Zhang, D. L. Michels, and C. Jiang, "Ships, splashes, and waves on a vast ocean," *ACM Transactions on Graphics (TOG)*, vol. 40, no. 6, pp. 1–15, 2021.
- [6] W. Zhang, J. Zhang, and T. Zhang, "Fast simulation method for ocean wave base on ocean wave spectrum and improved gerstner model with gpu," in *Journal of Physics: Conference Series*, vol. 787, no. 1. IOP Publishing, 2017, p. 012027.
- [7] L. J. Tomczak, "Gpu ray marching of distance fields," *Technical University of Denmark*, vol. 8, p. 3, 2012.
- [8] dr2. (2017-12-27) Tempting the mariner. [Online]. Available: <https://www.shadertoy.com/view/l12BzR>

- [9] ThomasSchander. (2017-08-22) Enscape cube. [Online]. Available: <https://www.shadertoy.com/view/4dSBDt>
- [10] afl_ext. (2017-03-09) Very fast procedural ocean. [Online]. Available: <https://www.shadertoy.com/view/MdXyzX>
- [11] Ships, splashes, and waves on a vast ocean (siggraph asia 2021). [Online]. Available: <https://www.youtube.com/watch?v=XcrOE4GTMec>
- [12] (2013) Smooth minimum. [Online]. Available: <https://iquilezles.org/articles/smin/>
- [13] (2024) Fractional brownian motion. [Online]. Available: https://en.wikipedia.org/wiki/Fractional_Brownian_motion
- [14] GDQuest. (2025) Fractal brownian motion (fbm). Godot Shaders. [Online]. Available: <https://godotshaders.com/snippet/fractal-brownian-motion-fbm/>
- [15] J. H. Lambert, *Photometria sive de mensura et gradibus luminis*. Leipzig, W. Engelmann, 1760.
- [16] J. F. Blinn, "Models of light reflection for computer synthesized pictures," *ACM SIGGRAPH Computer Graphics, Volume 11, Issue 2*, 1977.
- [17] N. Greene, "Environment mapping and other applications of world projections," *IEEE Computer Graphics and Applications*, 1986.
- [18] C. Schlick, "An inexpensive brdf model for physically-based rendering," *Computer Graphics Forum*, 1994.
- [19] Kelvin wake pattern. [Online]. Available: https://en.wikipedia.org/wiki/Kelvin_wake_pattern
- [20] Bow wake. [Online]. Available: https://en.wikipedia.org/wiki/Bow_wave